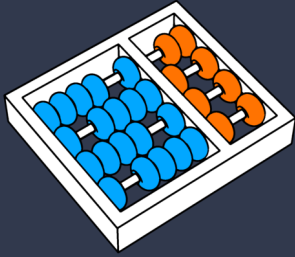




UNICAMP



# Arquiteturas de RAG

Prof. Dr. André Gomes Regino

regino@unicamp.br

# Agenda

01

## Pipeline do RAG

E onde estão as decisões críticas

02

## Chunking

Estratégias e trade-offs de segmentação

03

## Embeddings

Escolha do modelo e impacto na busca

04

## Retrieval

Top-k, similaridade e problemas clássicos

05

## Re-ranking

O que é e como funciona o refinamento

06

## Arquiteturas

Modelos Simples, Re-ranking e Híbridos

07

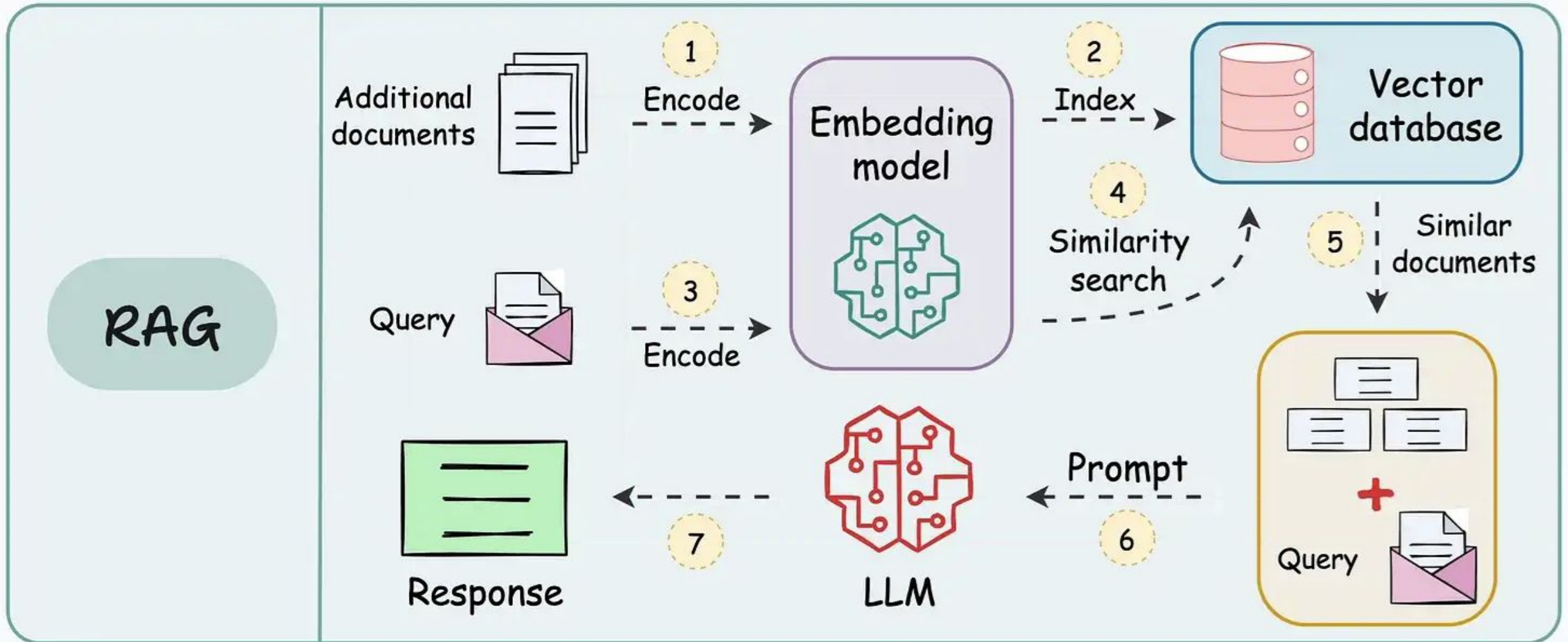
## Decisões de Design

Banco vetorial, atualização de dados e trade-offs finais

# Objetivos

1. **Identificar** cada etapa do pipeline RAG
2. **Escolher** uma estratégia de chunking adequada ao tipo de documento
3. **Comparar** modelos de embedding
4. **Implementar** retrieval com top-k e entender o impacto do valor de k
5. **Explicar** o papel do re-ranking e quando ele vale o custo adicional
6. **Distinguir** entre RAG simples, RAG com re-ranking e RAG híbrido

# Pipeline do RAG



# Onde estão as decisões?



# Chunking

# O que é Chunking?

## DEFINIÇÃO

Processo de dividir documentos longos em fragmentos menores antes da indexação vetorial. Cada fragmento (chunk) é indexado e recuperado de forma independente.

## Por que é necessário?



### Limites Técnicos e Semânticos

Modelos de embedding possuem limites de tokens (ex: 8.191 no text-embedding-3-small). Chunks excessivamente grandes degradam a qualidade semântica da representação.



### Granularidade de Busca

Um documento de 50 páginas indexado como um único vetor perde toda a precisão. O chunking permite encontrar a agulha no palheiro com precisão cirúrgica.

# Exemplo Visual

Documento original (2.000 tokens):

Parágrafo 1 ... Parágrafo 2 ... ... Parágrafo 40 ...

Após chunking (chunk\_size=500, overlap=50):

**Chunk 1**

0 – 500 tokens

**Chunk 2**

450 – 950 tokens

**Chunk 3**

900 – 1400 tokens

**Chunk 4**

1350 – ... tokens

sobreposição de 50 tokens entre chunks adjacentes



# Código

```
1 # pip install langchain-text-splitters==0.3.8
2 from langchain_text_splitters import RecursiveCharacterTextSplitter
3
4 splitter = RecursiveCharacterTextSplitter(
5     chunk_size=500,          # tamanho máximo em caracteres
6     chunk_overlap=50,        # sobreposição entre chunks
7     length_function=len,     # pode ser substituído por contagem de tokens
8 )
9
10 texto = "Documento longo aqui..."
11 chunks = splitter.split_text(texto)
12 print(f"{len(chunks)} chunks gerados")
13 print(f"Primeiro chunk: {chunks[0][:100]}...")
```

# Estratégias de Chunking

| Estratégia           | Descrição                                                                                         | Melhor para                                | Limitação                            |
|----------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------|--------------------------------------|
| Tamanho fixo         | Divide a cada N tokens/chars sem considerar estrutura                                             | Textos não estruturados, prototipagem      | Pode cortar frases no meio           |
| Com sobreposição     | Tamanho fixo + janela deslizando entre chunks                                                     | Maioria dos casos em produção              | Redundância (custo de indexação)     |
| Baseado em estrutura | Respeita parágrafos, seções, headers Markdown/HTML                                                | Documentos estruturados (manuais, artigos) | Chunks de tamanho variável           |
| Semântico            | Agrupa sentenças por similaridade semântica                                                       | Textos narrativos complexos                | Lento, requer embedding na indexação |
| Recursivo            | Tenta separadores em cascata ( $\backslash n \backslash n \rightarrow \backslash n \rightarrow$ ) | Uso geral                                  | Nenhuma limitação crítica            |

# Trade-offs

| Característica               | Chunks Pequenos ( $\leq 256$ tokens)                                    | Chunks Grandes ( $\geq 1024$ tokens)                           |
|------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------|
| Precisão da busca            | <b>Alta</b> — embedding representa tema específico                      | <b>Baixa</b> — embedding dilui o tema                          |
| Contexto para o LLM          | <b>Baixo</b> — pode faltar contexto ao redor                            | <b>Alto</b> — mais contexto disponível                         |
| Custos (Indexação e Geração) | Index: Mais chunks, cada um é barato<br>Geração: Menos tokens no prompt | Index: Menos chunks no total<br>Geração: Mais tokens no prompt |
| Risco Principal              | Resposta sem contexto suficiente                                        | Busca retorna chunk irrelevante                                |

# Embeddings

# O que são Embeddings?

## Definição

Representações vetoriais de texto em espaço de alta dimensão, onde textos semanticamente similares ficam geometricamente próximos.

## Conversão Dimensional

Texto → Vetor de N dimensões (ex.: 1.536 para *text-embedding-3-small*)

## Similaridade Semântica

A proximidade entre vetores representa similaridade de **significado**, não apenas palavras idênticas.

## Exemplo Prático:

"política de férias da empresa" → [0.23, -0.81, 0.44, ..., 0.12] (**Base**)

"quantos dias de folga tenho" → [0.21, -0.79, 0.46, ..., 0.11] ← **PRÓXIMO!**

"receita de bolo de chocolate" → [-0.54, 0.33, -0.71, ..., 0.88] ← **DISTANTE**

# Escolha de Embeddings

- qualidade semântica
- custo
- idioma (PT-BR!)
- domínio específico

```
1 # pip install sentence-transformers==3.3.1
2 from sentence_transformers import SentenceTransformer
3
4 model = SentenceTransformer("intfloat/multilingual-e5-large")
5 # Prefixo obrigatório para e5: "query: " e "passage: "
6 query_emb = model.encode("query: política de férias")
7 doc_emb = model.encode("passage: A empresa concede 30 dias de férias anuais.")
8
9 similarity = model.similarity(query_emb, doc_emb)
10 print(f"Similaridade: {similarity.item():.4f}")
```

# Impacto dos Embeddings

O embedding define o **teto de qualidade** do retrieval. Nenhuma etapa posterior compensa um embedding ruim.

## Métrica: Recall@k



Fração das queries em que o chunk correto está nos top-k recuperados.

**Recall@5  $\geq 0.80$**

Valor aceitável para produção

## Limite de Otimização



Se o desempenho estiver abaixo do esperado, a ação deve ser imediata.

**Recall@5  $< 0.70$**

**AÇÃO: Troque o embedding**

Antes de qualquer outra tentativa de otimização no pipeline.

# Retrieval



# Top-k

Impacto do valor de k no Retrieval

Top-k é a estratégia de recuperar os **k documentos** com maior similaridade ao vetor da query.

## k = 3

**Vantagem**

Rápido, prompt enxuto

**Risco**

Pode perder chunk essencial

**Ideal para**

Queries simples e diretas



Recomendado

## k = 5

**Vantagem**

Bom equilíbrio

**Risco**

Mínimo impacto

**Ideal para**

Padrão recomendado de produção

## k = 10

**Vantagem**

Alto recall

**Risco**

Mais ruído e custo de tokens

**Ideal para**

Queries complexas ou ambíguas

## k = 20+

**Vantagem**

Máximo recall possível

**Risco**

Lost in the Middle / Alucinação

**Ideal para**

Apenas com re-ranking posterior

# Top-k

Impacto do valor de k no Retrieval

```
1 # pip install langchain-community==0.3.20 langchain-openai==0.2.14  
  chromadb==0.6.x  
2 from langchain_community.vectorstores import Chroma  
3 from langchain_openai import OpenAIEmbeddings  
4  
5 vectorstore = Chroma(  
6     persist_directory="./meu_banco",  
7     embedding_function=OpenAIEmbeddings(model="text-embedding-3-small")  
8 )  
9  
10 retriever_k5 = vectorstore.as_retriever(  
11     search_type="similarity",  
12     search_kwargs={"k": 5} # ajuste aqui  
13 )  
14  
15 docs = retriever_k5.invoke("Qual o prazo de reembolso?")  
16 for i, doc in enumerate(docs):  
17     print(f"[{i+1}] {doc.page_content[:80]}...")  
18     print(f"    Fonte: {doc.metadata.get('source', 'N/A')}\n")
```

# Similaridade

Métricas de relevância entre vetores no Retrieval

A "**distância**" entre vetores define quais chunks são considerados relevantes.

## Métrica Principal: Cosine Similarity

Fórmula

$$\cos(\theta) = (\mathbf{A} \cdot \mathbf{B}) / (\|\mathbf{A}\| \cdot \|\mathbf{B}\|)$$

Intervalo

$[-1, 1]$

Quando usar

**Padrão recomendado para sistemas RAG**

## Por que é o padrão?



Mede o **ângulo** entre vetores, ignorando a magnitude absoluta.



Ideal para documentos de **tamanhos diferentes** que abordam o mesmo tema.

# Problema Clássico

Limitações do retrieval por similaridade vetorial

O retrieval recupera o que é **semanticamente próximo**, não necessariamente o que **responde** à query.

| Causa                       | Exemplo                                               |
|-----------------------------|-------------------------------------------------------|
| Vocabulário diferente       | Query: "prazo de entrega" vs. Doc: "SLA de expedição" |
| Query muito curta           | "reembolso?" — sem contexto suficiente para embedding |
| Chunk errado recuperado     | Chunk com a palavra certa, mas contexto errado        |
| k muito pequeno             | Chunk correto está na posição 6, k=5                  |
| Embedding não especializado | Modelo genérico em domínio técnico (ex.: jurídico)    |

# Re-ranking

# O que é Re-ranking?

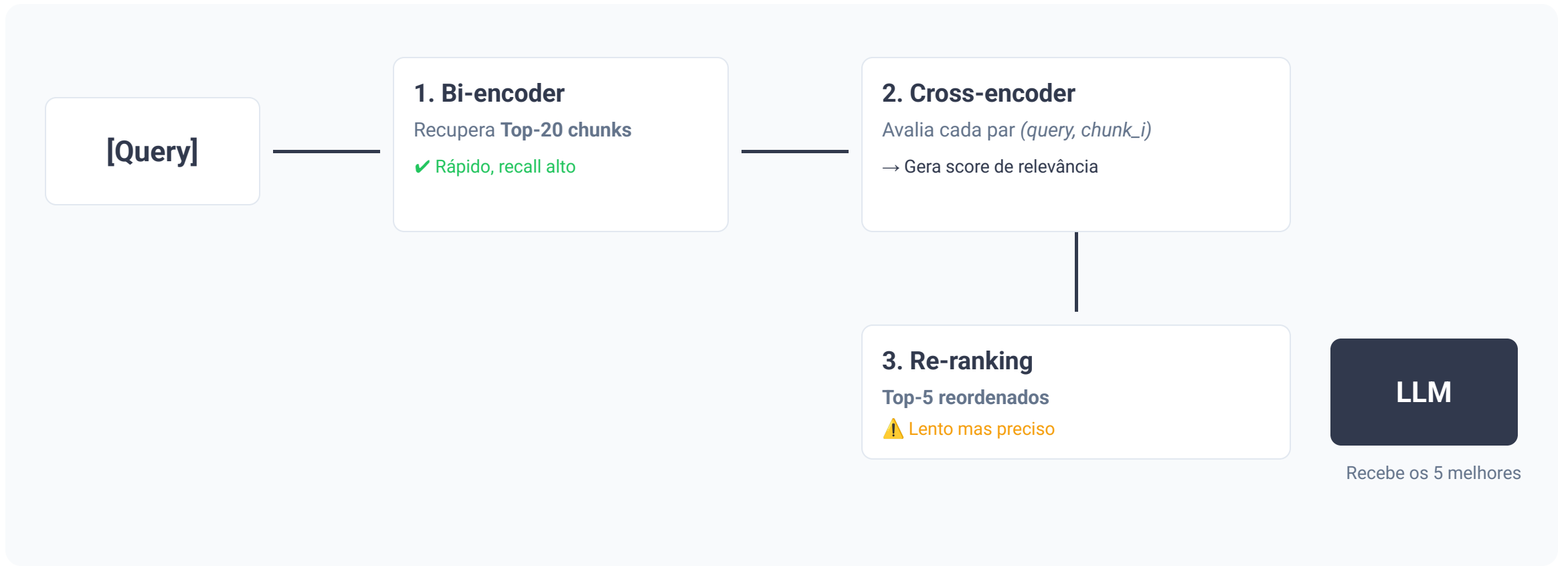
Re-ranking é uma **segunda etapa de seleção** que reordena os chunks com um modelo mais preciso (geralmente um **cross-encoder**).

| Característica | Bi-encoder (Retriever)              | Cross-encoder (Re-ranker)        |
|----------------|-------------------------------------|----------------------------------|
| Input          | Query e documento separados         | Query + documento juntos         |
| Velocidade     | Rápida (busca por índice)           | Lenta (inference por par)        |
| Precisão       | Média                               | Alta                             |
| Escala         | Sobre toda a base (milhões de docs) | Apenas sobre top-k (ex: 20 docs) |

# Como Funciona?

- recupera top-k (ex: 20)
- modelo de re-ranking escolhe os melhores (ex: 5)
- Benefícios
  - maior precisão
  - melhor contexto para o LLM

# Fluxo





# RAG Simples vs Avançado

# RAG Simples

Query → Embedding → Busca Vetorial (top-k) → Prompt → LLM → Resposta



## Quando usar

Prototipagem, bases pequenas (<10k docs), queries simples e diretas, latência crítica (<1s).



## Pipeline direto

Sem re-ranking, sem busca híbrida.

## Vantagens

- ✓ Fácil de implementar e depurar
- ✓ Latência mínima (uma chamada de embedding + busca)

## Limitações & Riscos

- ⚠ Qualidade do retrieval limita a qualidade da resposta
- ⚠ Sensível ao valor de k e à qualidade do embedding

# RAG como Re-ranking

[Query] → Embedding → Busca Vetorial (top-20) → **Cross-Encoder Re-ranking** → Top-5 refinados → Prompt → LLM → Resposta

## Vantagens

- ✓ Precisão significativamente maior que RAG simples
- ✓ Compensa limitações do embedding (bi-encoder)

## Limitações & Custos

- ⚠ Latência adicional: +100–600ms (Local/API)
- ⚠ Custo extra se usar APIs externas (ex.: Cohere)



## Quando usar

Bases médias/grandes, queries ambíguas, domínios especializados e cenários onde a precisão é o requisito prioritário sobre a latência.

# RAG Híbrido

- combina:
  - busca vetorial
  - busca por palavra-chave (BM25)
- ✓ melhor recall
- ✓ robusto

# Decisões de Design

# Escolha do Banco Vetorial

## Critérios de Seleção

⚡ Performance  Escalabilidade  Custo  Facilidade de uso

### Protótipo

## Chroma

Ideal para desenvolvimento rápido com zero configuração.

### Produção c/ Postgres

## pgvector

Aproveite a infraestrutura existente do time sem novos silos.

### Produção Cloud

## Pinecone / Qdrant

Soluções Zero Ops escaláveis para ambientes em nuvem.

### Alta Escala Local

## FAISS + Metadados

Volumes massivos de dados com gerenciamento externo de metadados.

### Filtragem Complexa

## Weaviate / Qdrant

Necessidade de buscas híbridas e filtros avançados por metadados.

# Atualização dos Dados

| Estratégia             | Quando re-indexa                    | Latência         | Complexidade | Caso de uso                                |
|------------------------|-------------------------------------|------------------|--------------|--------------------------------------------|
| Batch completo         | Periodicamente (ex: toda noite)     | Alta (horas)     | Baixa        | Dados que mudam pouco (manuais, políticas) |
| Batch incremental      | Apenas documentos novos/modificados | Média (minutos)  | Média        | Maioria dos casos em produção              |
| Streaming / tempo real | A cada atualização de documento     | Baixa (segundos) | Alta         | Dados críticos (preços, estoque, notícias) |

# Custo vs Qualidade

## Nível 1 — Simples

### Stack:

text-embedding-3-small + Chroma + k=5 + gpt-4o-mini

### Objetivo:

Baseline. Mede Recall@5 e qualidade das respostas.

## Nível 2 — Re-ranking

### Adição:

+ [FlashrankRerank](#) (local, grátis)

### Gatilho:

Se Recall@5 < 0.80 ou respostas imprecisas.

## Nível 3 — Híbrido

### Adição:

+ [BM25Retriever](#) + [EnsembleRetriever](#)

### Gatilho:

Se queries técnicas com termos exatos falham.

## Nível 4 — Produção

### Otimização Fina:

[Embedding multilíngue](#) + [chunking semântico](#) + [Cohere Rerank](#)

### Uso:

Para produção com requisitos de qualidade altos.

**Princípio:** evoluir incrementalmente. Não pule para o Nível 4 sem validar que o Nível 1 já não é suficiente.



# Take Home Message



## Design, não receita

RAG é um espaço de parâmetros ajustáveis (qualidade, custo, latência).



## Chunking é a base

O maior determinante de qualidade. Revise a divisão antes de trocar o modelo.



## O Embedding é o teto

Nada compensa um embedding ruim. Meça Recall@k primeiro.



## Top-k & Re-ranking

K=5 é um bom início. Re-ranking oferece alto ganho de precisão (+100ms).



## Híbrido é robusto

Use vetorial + BM25 para termos técnicos, siglas e códigos.



## Evolução incremental

Meça sempre antes de adicionar complexidade.

## Referências da Aula

**Gao et al. (2023)**. RAG for LLMs: A Survey. arXiv:2312.10997

**Nogueira & Cho (2019)**. Passage Re-ranking with BERT. arXiv:1901.04085

**Gao et al. (2022)**. HyDE: Zero-Shot Dense Retrieval. arXiv:2212.10496

**Cormack et al. (2009)**. Reciprocal Rank Fusion. SIGIR 2009

**Muennighoff et al. (2023)**. MTEB Benchmark. arXiv:2210.07316

*"RAG é um espaço de design, não uma receita."*